AI Travel Itinerary Planner: Project Documentation

Project Description

This project is a web application that makes a day-by-day travel plan. The user enters the destination, travel dates, interests, budget and pace, and the app asks an AI model to write the plan. Every day has morning, afternoon and evening activities with notes on transport, food and approximate cost. A logged-in user can save a plan, open it later, rename it, delete it or download it as a PDF. On screen the app is called Trip Genius AI.

The AI part uses the Groq API with the `openai/gpt-oss-120b` model. The frontend is built with React and TypeScript, the backend with FastAPI, and Supabase handles login and the database. The frontend is hosted on Vercel and the backend on Render. API keys are stored in environment variables and each user's data is protected with Row Level Security.

Scenarios

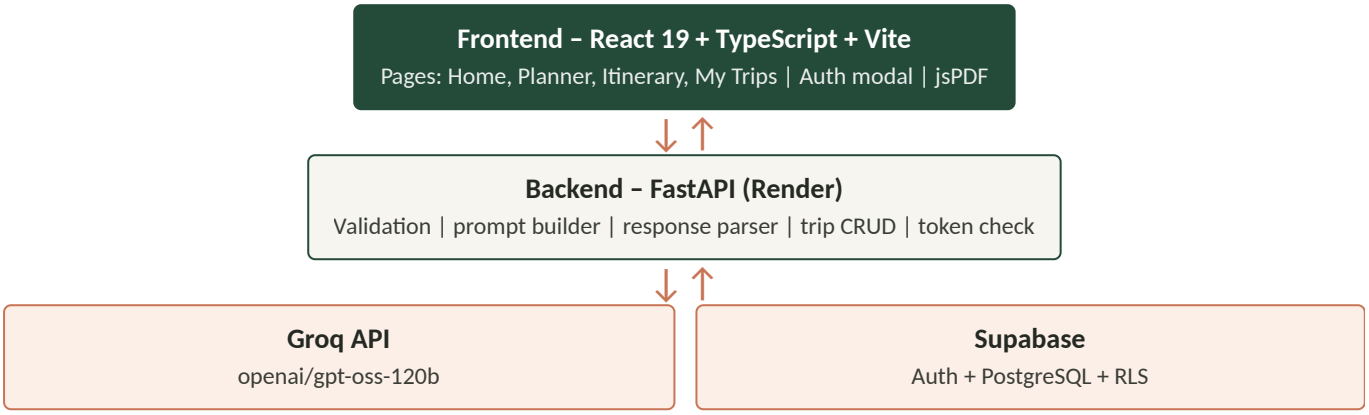
Scenario 1: Budget student trip: A student enters Manali, 3 days, interests Nature + Adventure + Food, budget tier "budget". The system returns three days of low-cost treks, local cafes and public transport hints.

Scenario 2: Cultural holiday: A traveller selects Kyoto for 5 days with Culture + Food and a relaxed pace. The plan has 2–3 unhurried activities per day with temple visits and market food suggestions.

Scenario 3: Save and share: A professional logs in, saves an Amalfi Coast itinerary, finds it later in My Trips and downloads the PDF for offline use during the trip.

Technical Architecture

The system has three parts. The React frontend takes the input and shows the result. The FastAPI backend checks the data, builds the prompt and reads the AI output. Supabase stores the users and their saved trips.



ER Diagram

The database has two application tables linked to Supabase `auth.users`: **trips** (id, user_id, title, destination, start_date, end_date, budget, travel_style, preferences JSONB, status, timestamps) and **itineraries** (id, trip_id UNIQUE, user_id, itinerary_data JSONB, timestamps). One user has many trips; one trip has exactly one itinerary. Both tables enforce `auth.uid() = user_id` through RLS. See *3. Project Design Phase / Solution Architecture* for the diagram.

Pre-requisites

- React and TypeScript knowledge – react.dev, typescriptlang.org
- Python and FastAPI – fastapi.tiangolo.com
- Groq API – console.groq.com/docs
- Supabase Auth and PostgreSQL – supabase.com/docs
- Git and GitHub – git-scm.com/doc
- Deployment on Vercel and Render – vercel.com/docs, render.com/docs

Project Workflow

Milestone 1: Model Selection and Architecture

- Activity 1.1: Create a Groq API key and store it in the backend `.env` as `GROQ_API_KEY`.
- Activity 1.2: Select `openai/gpt-oss-120b` on Groq for strong reasoning and fast inference (configurable with `GROQ_MODEL`).
- Activity 1.3: Define the architecture: React, FastAPI, Groq and Supabase.
- Activity 1.4: Set up environments: Node.js with Vite, Python venv, `pip install -r requirements.txt`.

Milestone 2: Backend Core Functionality

- Activity 2.1: Define Pydantic schemas (`ItineraryGenerateRequest`: destination, dates, interests, budget, pace) with validators for blank destination, empty interests, date order and a 30-day limit.
- Activity 2.2: Write `build_groq_prompts()` that creates a system prompt demanding strict JSON (days, title, notes, activities) and a user prompt with the traveller's details.
- Activity 2.3: Call Groq (temperature 0.7), parse JSON, verify the exact number of days and at least three activities per day, and map timeout, rate-limit and invalid-output failures to HTTP errors.

Milestone 3: API Routes and Database

- Activity 3.1: Expose `GET /health`, `POST /api/itineraries/generate` and the `/api/trips` routes (POST, GET list, GET by id, PUT, PATCH, DELETE).
- Activity 3.2: Verify the Bearer token with Supabase in `deps.py` before any trip operation.
- Activity 3.3: Run `schema.sql` to create trips and itineraries tables, indexes and RLS policies.

Milestone 4: Frontend Development

- Activity 4.1: Build pages (Home, Planner, Itinerary, My Trips), the auth modal and the shared layout with the forest-green / terracotta theme.
- Activity 4.2: Implement the planner form with interest chips, budget and pace selectors and validation messages.
- Activity 4.3: Connect `api.ts` and `supabase.ts`; manage session with the `useAuth` hook; render itineraries day by day.
- Activity 4.4: Implement PDF export with jsPDF including text wrapping and page breaks.

Milestone 5: Testing

- Activity 5.1: Write 34 pytest tests covering health, CORS, itinerary generation (success, demo, timeout, rate limit, invalid JSON, missing days, validation) and trips (auth, CRUD, isolation, DB errors).
- Activity 5.2: Manually test the planner, auth, save, My Trips and PDF flows in the browser.

Milestone 6: Deployment

- Activity 6.1: Deploy the backend to Render with `gunicorn app.main:app -k uvicorn.workers.UvicornWorker --bind 0.0.0.0:$PORT` and set environment variables and CORS origins.
- Activity 6.2: Deploy the frontend to Vercel (`npm run build`, output `dist`) with `VITE_API_URL` pointing to Render.
- Activity 6.3: Test production sign-up, generation, save and PDF export.

Milestone 7: Conclusion

Through this project I built a complete full-stack application with AI itinerary generation, login, separate data for every user, PDF export and online deployment. In future I want to add maps, weather, live prices, shared trips and editing of single activities.

> **Demo login:** 20programmer28@gmail.com / Login@123 **Live:** https://travel-itinerary-planner-sage.vercel.app
> **Code:** https://github.com/ProGraMMer-Parth/travel-itinerary-planner